

INSTITUTO POLITÉCNICO DE BRAGANÇA
Internet of Things

IoT Alarm System

Motion detection with ESP32, control via Node-RED
and encrypted MQTT communication (RSA)

AUTHORS

Luis Henrique da Silva Resende - a67608
José Lês inadmi - a58331

June 2026

1. Introduction

This report describes the IoT Alarm System project, developed within the Internet of Things course at the Instituto Politécnico de Bragança. The system implements a simple alarm based on an ESP32 microcontroller, remotely controlled through a web interface served by Node-RED. Communication between the interface and the device is done via MQTT, and the user's PIN is protected by RSA encryption.

The project brings together three complementary pieces: the physical device (ESP32 with sensor and actuators), the orchestrator (Node-RED, which serves the interface, encrypts commands, and bridges the broker), and the user interface (a numeric keypad built in Vue). The work focuses both on the integration of these pieces and on the messaging pattern adopted and PIN protection.

The project source code is available in the link: <https://github.com/luishresende/iot-alarm-esp32>.

1.1. Objectives

- Detect motion and signal a local alarm, both audible and visual.
- Arm and disarm the alarm remotely by entering a PIN.
- Allow the PIN to be changed from the interface itself.
- Ensure PIN confidentiality in transit through encryption.
- Support multiple users/sessions over a simple MQTT infrastructure.

2. System Architecture

The architecture is organized into three blocks connected by a public MQTT broker (broker.mqtt-dashboard.com). The user interacts with the keypad in the browser; Node-RED receives the command, encrypts it, and publishes it; the ESP32 receives the command, decrypts it, acts on the alarm, and returns the response.

The message exchange follows a pattern of **shared queue on the way in and temporary topic on the way back**. Commands from all users enter a single topic — the shared queue — which the ESP32 subscribes to. Responses are published by the ESP32 on a session-exclusive topic, identified by a **UUID** generated in the browser. Thus, although everyone shares the same input channel, each user receives only the response to their own request.

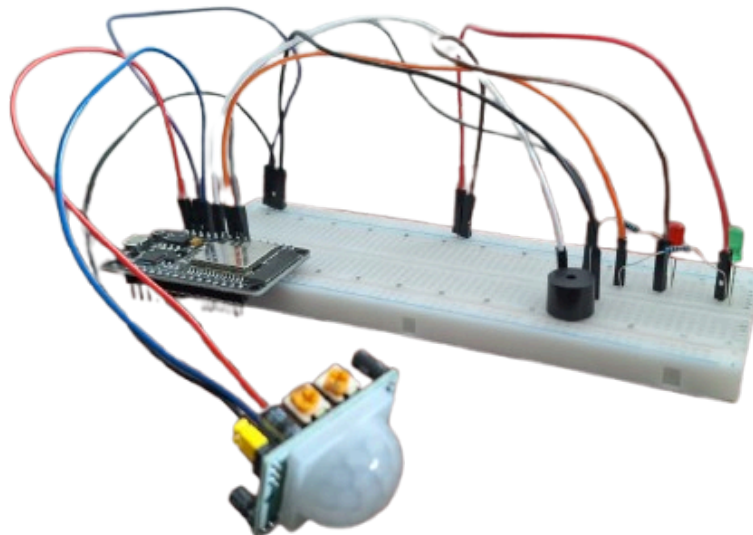
The three components are:

- **Device (ESP32)**. Runs the firmware, reads the motion sensor, and activates the actuators (buzzer and LEDs). Generates the RSA key pair and maintains the alarm state.
- **Orchestrator (Node-RED)**. Serves the interface (Dashboard 2.0), encrypts commands with the ESP32's public key, publishes them to the queue, and routes responses to the correct session.
- **Interface (Vue)**. Numeric keypad in the browser; collects the PIN, generates the session UUID, and displays the result to the user.

3. Hardware (the alarm)

The device is an ESP32 with integrated Wi-Fi. Connected to it are a motion sensor (PIR) as input and three actuators as output: a buzzer and two LEDs (green and red). All pins operate in digital mode.

Component	Pin	Direction	Function
ESP32	—	—	Microcontroller; Wi-Fi and MQTT client.
PIR Sensor (motion)	D13	Input	Detects motion; with the system armed, triggers the alarm.
Buzzer	D14	Output	Emits the alarm sound.
Green LED	D26	Output	Indicates the system is disarmed / at rest.
Red LED	D25	Output	Indicates the system is armed and in alarm state.



The actuator behavior is summarized in three states. Once triggered, the alarm latches and is only silenced by a valid PIN. The buzzer and red LED blinking is non-blocking (based on `millis()`), so the ESP32 continues processing MQTT messages — and therefore the disarm command — even during the alarm.

State	Green LED	Red LED	Buzzer	Description
Idle	On	Off	Silent	System disarmed; does not react to motion.
Armed	Off	On (steady)	Silent	Monitoring; motion triggers the alarm.
Alarm	Off	Blinking	Blinking (200 ms)	Motion detected; persists until a valid PIN is entered.

4. ESP32 Firmware

The firmware (Arduino/ESP32) is organized around the `setup()` and `loop()` functions. On startup, it configures the pins, loads the stored PIN, generates the RSA key pair, and connects to Wi-Fi and the broker. In the main loop, it maintains the MQTT connection, evaluates the sensor, updates the actuators, and periodically publishes the public key.

4.1. Libraries Used

Library	Function
WiFi.h	Connects the ESP32 to the local Wi-Fi network.
PubSubClient	MQTT client: publishes and subscribes to topics on the broker.
ArduinoJson	Serializes and deserializes message JSON.
Preferences	Stores the PIN in non-volatile memory (NVS).
mbed TLS	Generates RSA keys, decrypts (OAEP-SHA1), and encodes/decodes in Base64.

4.2. Keys, Persistence and Processing

On initialization, the ESP32 generates a **2048-bit** RSA key pair (exponent 65537) using the **OAEP-SHA1** padding scheme, and publishes the public key (in PEM format) to the topic `.../Crypto/PublicKey/` every 5 seconds. The PIN is stored in NVS via the Preferences library, defaulting to **123456** on first boot.

Upon receiving a message from the queue, the ESP32 decrypts it with its private key, interprets the JSON, and dispatches the action. For a **login**, it compares the received PIN with the stored one and toggles between armed and disarmed; on disarming, it immediately silences any ongoing alarm. For an **update_password**, it validates the old password and the length of the new one (between 1 and 6 digits) and saves it to NVS. In both cases it returns a response on the session's temporary topic.

5. Dashboard (Node-RED and Vue)

The interface is served by **Node-RED Dashboard 2.0** (based on Vue/Vuetify). The keypad is a Vue component defined in a **ui-template** node, which reacts to Node-RED responses via a **watch** on the `msg` variable and sends commands with `this.send()`.

Each confirmation generates a session identifier with `crypto.randomUUID()` (with a fallback mechanism for environments without that API). The interface has two modes: normal mode, where the entered PIN is sent as a login request; and password-change mode, activated by entering the code `000000`, which prompts for the current password, the new one, and confirmation. During the change, the screen displays "Waiting..." and blocks the keypad until the response arrives.

On the Node-RED side, the flow combines native and extension nodes:

Node	Package	Function
ui-template *	@flowfuse/node-red-dashboard	Dashboard 2.0 (Vue); hosts the keypad.

Node	Package	Function
rsa-encrypt	sense-rsa (node-rsa)	Encrypts the command with RSA (OAEP-SHA1).
mqtt in / out	Node-RED (native)	Connects to broker; publishes and subscribes.
function	Node-RED (native)	Assembles the payload and subscribes/unsubscribes from the {uuid} topic.
inject / debug	Node-RED (native)	Test triggering and debugging.

6. MQTT Communication

All communication relies on the public broker `broker.mqtt-dashboard.com`, under the prefix `IPB/IoT/AlarmProject/`. Three topics are used, with distinct Quality of Service (QoS) levels.

Topic	Direction	QoS	Content
<code>.../Crypto/PublicKey/</code>	ESP32 → Node-RED	1	ESP RSA public key, published every 5 s.
<code>.../Notifications/Personal/</code>	Node-RED → ESP32	1	Encrypted commands; shared queue.
<code>.../Notifications/Personal/{uuid}</code>	ESP32 → user	1	Operation response; temporary topic per session.

Response routing uses **dynamic subscription**: before publishing a command, Node-RED subscribes to the topic `.../Personal/{uuid}`; after delivering the response to the session, it cancels that subscription. This UUID-based correlation is what allows serving multiple users over a single input channel.

7. Security

PIN confidentiality is ensured by RSA asymmetric encryption. The ESP32 generates the key pair and discloses the public key; Node-RED uses that key to encrypt the command before publishing it. Since only the ESP32 holds the private key, only it can decrypt the PIN — which therefore never travels in plaintext through the broker.

Despite this protection, the work has security limitations that must be noted, especially due to the use of a public shared broker. These limitations are detailed in the following section.

8. Operation Flows

8.1. Login (arm / disarm)

1. The user enters the PIN and confirms; the browser generates a **UUID** and sends { **action: login, pin, uuid** } to Node-RED.
2. Node-RED subscribes to the topic `.../Personal/{uuid}` and encrypts the command with the public key (RSA).
3. Publishes the encrypted command to the shared queue `.../Notifications/Personal/`

4. The ESP32, subscribed to the queue, decrypts the command, validates the PIN, and toggles between armed and disarmed.
5. The ESP32 responds at `.../Personal/{uuid}` with `{ success, armed }`.
6. Node-RED delivers the response to the session and cancels the topic subscription.

If the PIN is invalid, the ESP32 responds with `{ success: false, message: "Invalid PIN" }` and the system state does not change.

8.2. Password Change

1. The user enters the code `000000`, which activates password-change mode.
2. The interface prompts, in this order, for the current password, the new password, and its confirmation.
3. If the new password and confirmation match, the browser sends `{ action: update_password, old_password, new_password, uuid }`.
4. The ESP32 validates the old password and the length of the new one (1 to 6 digits), saves it to NVS, and responds.
5. While waiting, the screen shows “Waiting...”; upon receiving the response, it displays the result.

8.3. Motion Trigger

With the system armed, motion detection by the PIR latches the alarm state. The red LED and buzzer blink in 200 ms cycles, non-blocking, and remain so until a valid PIN disarms the system.

9. Limitations and Future Work

- Public broker: anyone can subscribe to the topics. A private broker with authentication is recommended.
- Unencrypted connection (port 1883): enable MQTT over TLS (MQTTS).
- Wi-Fi credentials and broker address hardcoded in the firmware: externalize the configuration.

10. Conclusion

The system meets the proposed objectives: it detects motion, signals the alarm, allows remote arming and disarming, supports password changes, and protects the PIN in transit. All of this on a simple MQTT architecture, with a clear pattern of shared queue on input and per-session temporary topics on output.

The identified limitations, especially the use of a public broker without TLS, point to direct paths for improvement, focused on strengthening transport security and configuration management. Even so, the project functionally demonstrates the core concepts of the course: device integration, message-based communication, and protection of sensitive data.